11 MAGGIO 2026

Documentazione dell'implementazione Java + JML del sistema ascensore

MAFFEIS RICCARDO 1085706

Tema scelto	Ascensore a più piani con un'unica cabina
Obiettivo	Realizzazione, specifica e verifica del nucleo logico del sistema modellato in ASM, con interfaccia web dimostrativa
Metodo	Implementazione Java annotata con contratti JML, verifica statica tramite ESC/OpenJML, test JUnit, test Selenium e Continuous Integration
Proprietà verificate	Invarianti di sicurezza, precondizioni, postcondizioni, invarianti di ciclo, comportamento operativo e integrazione dell'interfaccia web con il nucleo Java
Esito	Il nucleo logico risulta coerente con il modello ASM e verificato tramite specifiche JML, controllo statico, test automatici JUnit/Selenium e pipeline GitHub Actions

Sommario

1. Introduzione 2
2. Differenze rispetto al modello ASM 2
3. Struttura del codice 3
4. Classe Ascensore 4
5. Rappresentazione delle richieste attive 5
6. Invarianti JML della classe 5
6.1 Vincoli sullo stato numerico 5
6.2 Vincoli di non nullità 6
6.3 Invarianti di sicurezza sul movimento 6
6.4 Invarianti sul guasto 6
6.5 Invarianti sul sovraccarico 6
6.6 Invariante sul timer 6
7. Costruttore e stato iniziale 7
8. Metodi principali 8
8.1 Metodi di osservazione 8
8.2 Gestione delle richieste 8
8.3 Gestione persone e sovraccarico 9
8.4 Gestione guasto e timer 10
8.5 Gestione porte, servizio del piano e stato di attesa 10
8.6 Ricerca richieste, scelta direzione e movimento 11
9. Classe InputAscensore 13
10. Classe ControlloreAscensore 14
11. Interfaccia web dimostrativa 14
12. Test JUnit 16
13. Test Selenium 16
14. Verifica tramite ESC/OpenJML 17
15. Conclusione 18

1. Introduzione

Questa documentazione descrive l'implementazione Java del nucleo logico del sistema ascensore modellato precedentemente mediante ASM.

L'obiettivo dell'implementazione Java non è sostituire il modello ASM, ma realizzare una versione eseguibile e verificabile del comportamento principale dell'ascensore. La classe Ascensore rappresenta il nucleo operativo del sistema ed è annotata mediante JML per specificare formalmente proprietà, vincoli e contratti dei metodi principali.

Alla classe Ascensore si affiancano la classe *InputAscensore*, usata per rappresentare gli input esterni del sistema, e la classe *ControlloreAscensore*, che coordina l'evoluzione dell'ascensore a ogni passo logico.

La specifica JML è stata utilizzata per definire invarianti di classe, precondizioni, postcondizioni e proprietà sui cicli. La verifica statica tramite ESC/OpenJML consente di controllare la coerenza del codice rispetto alle proprietà dichiarate, mentre i test JUnit verificano il comportamento operativo del nucleo Java in scenari concreti.

Oltre al nucleo logico, il progetto include una semplice interfaccia web dimostrativa basata su HttpServer, utilizzata per osservare dinamicamente il comportamento dell'ascensore e per interagire con il sistema in modalità automatica o manuale. L'interfaccia utilizza direttamente le classi del nucleo Java e non introduce una nuova logica di controllo.

La parte web è stata inoltre verificata tramite test Selenium, che controllano l'integrazione tra pagina web, server HTTP e nucleo logico Java.

2. Differenze rispetto al modello ASM

Il modello ASM descrive il comportamento dell'ascensore come una macchina astratta a stati. In ASM il sistema è definito tramite domini, funzioni monitored, funzioni controlled e regole di transizione.

L'implementazione Java organizza invece la logica del sistema in metodi separati. Ogni metodo rappresenta una parte del comportamento del modello ASM.

Nel modello ASM gli input esterni erano rappresentati da funzioni monitorate:

```
monitored chiamataPianoSu: Piano -> Boolean
monitored chiamataPianoGiu: Piano -> Boolean
monitored pulsanteInterno: Piano -> Boolean
monitored eventoGuasto: Boolean
```

```
monitored personeEntrate: Integer
monitored personeUscite: Integer
```

Nel codice Java questi input sono raccolti nella classe *InputAscensore* e poi interpretati dal *ControlloreAscensore*.

Ad esempio, una richiesta o un evento vengono impostati sull'oggetto di input tramite metodi come:

```
input.setRichiestaInterna(piano);
input.setChiamataSalita(piano);
input.setChiamataDiscesa(piano);
input.setEventoGuasto(true);
input.setPersoneEntrate(valore);
input.setPersoneUscite(valore);
```

Il controllore usa poi tali informazioni per invocare i metodi della classe Ascensore.

Inoltre, mentre nel modello ASM il comportamento complessivo è coordinato da una regola principale, nella versione Java tale ruolo è svolto dal metodo eseguiPasso(InputAscensore input) della classe *ControlloreAscensore*.

3. Struttura del codice

Il codice è organizzato principalmente nel *package progetto*, che contiene il nucleo logico del sistema. A questo si aggiungono il *package web*, dedicato all'interfaccia dimostrativa, e il *package test*, dedicato ai test automatici.

Le classi principali sono:

- *Ascensore*, che contiene lo stato interno e i metodi fondamentali dell'ascensore;
- *InputAscensore*, che rappresenta gli input esterni ricevuti in un passo logico;
- *ControlloreAscensore*, che coordina l'evoluzione del sistema;
- *ControlloreAscensoreTest*, che contiene i test JUnit;
- *AscensoreHttpServer*, che realizza una semplice interfaccia web dimostrativa;
- *AscensoreWebSeleniumTest*, che contiene i test Selenium sull'interfaccia web.

Le enumerazioni utilizzate sono *Direzione*, *StatoCabina*, *StatoPorte* e *StatoErrore*.

4. Classe Ascensore

La classe *Ascensore* contiene lo stato principale del sistema e i metodi che implementano la logica dell'ascensore.

Le costanti principali sono:

```
public static final int PIANO_MINIMO = -1;
public static final int PIANO_MASSIMO = 4;
public static final int NUMERO_PIANI = 6;
public static final int CAPACITA_MASSIMA = 8;
public static final int TIMER_MASSIMO = 10;
```

Queste costanti definiscono:

- il piano minimo raggiungibile;
- il piano massimo raggiungibile;
- il numero totale di piani;
- la capacità massima della cabina;
- il valore massimo del timer di ripristino dopo un guasto.

Essendo dichiarate come public static final, tali costanti sono fissate direttamente dal linguaggio Java e non necessitano di ulteriori invarianti JML.

Lo stato interno dell'ascensore è rappresentato dai seguenti attributi:

```
private /*@ spec_public @*/ int pianoCorrente;
private /*@ spec_public @*/ int timer;
private /*@ spec_public @*/ int numeroPersone;

private /*@ spec_public @*/ StatoCabina statoCabina;
private /*@ spec_public @*/ StatoPorte statoPorte;
private /*@ spec_public @*/ Direzione direzione;
private /*@ spec_public @*/ StatoErrore statoErrore;

private /*@ spec_public @*/ final boolean[] richiesteAttive;
```

Gli attributi sono dichiarati private per proteggere lo stato interno della classe e impedire modifiche dirette non controllate dall'esterno.

L'annotazione JML:

```
/*@ spec_public @*/
```

consente comunque di usare questi campi nelle specifiche JML pubbliche.

5. Rappresentazione delle richieste attive

Nel modello ASM le richieste erano rappresentate come funzione:

```
richiesteAttive: Piano -> Boolean
```

Nel codice Java sono rappresentate tramite un array:

```
private final boolean[] richiesteAttive;
```

Poiché in Java gli array partono da indice 0, mentre i piani dell'ascensore vanno da -1 a 4, viene utilizzato il metodo:

```
indiceDelPiano(int piano)
```

Il metodo converte il numero del piano nell'indice dell'array.

Il contratto JML del metodo è:

```
//@ requires pianoValido(piano);
//@ ensures 0<=\result && \result<NUMERO_PIANI;
//@ ensures \result == piano – PIANO_MINIMO;
```

Questo garantisce che ogni piano valido venga convertito in un indice valido dell'array.

6. Invarianti JML della classe

La classe Ascensore definisce diversi invarianti JML. Gli invarianti rappresentano proprietà che devono essere sempre vere per ogni oggetto valido della classe.

6.1 Vincoli sullo stato numerico

```
//@ public invariant PIANO_MINIMO<=pianoCorrente && pianoCorrente<=PIANO_MASSIMO;
//@ public invariant 0<=timer && timer<=TIMER_MASSIMO;
//@ public invariant numeroPersone>=0;
```

Questi invarianti garantiscono che:

- il piano corrente sia sempre compreso tra il piano minimo e il piano massimo;
- il timer sia sempre compreso tra 0 e TIMER_MASSIMO;
- il numero di persone non sia mai negativo.

6.2 Vincoli di non nullità

```
//@ public invariant statoCabina != null;
//@ public invariant statoPorte != null;
//@ public invariant direzione != null;
//@ public invariant statoErrore != null;
//@ public invariant richiesteAttive != null;
//@ public invariant richiesteAttive.length == NUMERO_PIANI;
```

Questi invarianti garantiscono che gli stati dell'ascensore siano sempre definiti e che l'array delle richieste sia correttamente inizializzato.

6.3 Invarianti di sicurezza sul movimento

```
//@ public invariant statoCabina == StatoCabina.IN_MOVIMENTO ==> statoPorte == StatoPorte.CHIUSE;
//@ public invariant statoCabina == StatoCabina.IN_MOVIMENTO ==> statoErrore == StatoErrore.NESSUNO;
```

Questi invarianti formalizzano due proprietà fondamentali:

- la cabina può muoversi solo con le porte chiuse;
- la cabina può muoversi solo se non sono presenti errori.

6.4 Invarianti sul guasto

```
//@ public invariant statoErrore == StatoErrore.GUASTO ==> statoCabina == StatoCabina.BLOCCATA;
//@ public invariant statoErrore == StatoErrore.GUASTO ==> statoPorte == StatoPorte.CHIUSE;
//@ public invariant statoErrore == StatoErrore.GUASTO ==> direzione == Direzione.NESSUNA;
```

Quando il sistema è in stato GUASTO, la cabina deve essere bloccata, le porte devono essere chiuse e la direzione deve essere annullata.

6.5 Invarianti sul sovraccarico

```
//@ public invariant statoErrore == StatoErrore.OVERLOAD ==> statoCabina == StatoCabina.BLOCCATA;
//@ public invariant statoErrore == StatoErrore.OVERLOAD ==> statoPorte == StatoPorte.APERTE;
//@ public invariant statoErrore == StatoErrore.OVERLOAD ==> direzione == Direzione.NESSUNA;
```

Quando il sistema è in stato OVERLOAD, la cabina deve essere bloccata, le porte devono essere aperte e la direzione deve essere annullata.

6.6 Invariante sul timer

```
//@ public invariant timer>0 ==> statoErrore == StatoErrore.GUASTO;
```

Questo invariante stabilisce che il timer può essere positivo solo durante uno stato di guasto.

7. Costruttore e stato iniziale

Il costruttore inizializza l'ascensore nello stato iniziale:

public Ascensore()

lo stato iniziale è:

```
pianoCorrente = 0
timer = 0
numeroPersone = 0
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
statoErrore = NESSUNO
richiesteAttive = array di 6 elementi
```

Le postcondizioni JML garantiscono che l'oggetto venga creato in uno stato coerente:

```
//@ ensures pianoCorrente == 0;
//@ ensures timer == 0;
//@ ensures numeroPersone == 0;
//@ ensures statoCabina == StatoCabina.FERMA;
//@ ensures statoPorte == StatoPorte.CHIUSE;
//@ ensures direzione == Direzione.NESSUNA;
//@ ensures statoErrore == StatoErrore.NESSUNO;
//@ ensures richiesteAttive != null;
//@ ensures richiesteAttive.length == NUMERO_PIANI;
```

8. Metodi principali

8.1 Metodi di osservazione

La classe contiene diversi metodi *pure*, utilizzati per leggere lo stato senza modificarlo:

```
getPianoCorrente()
getTimer()
getNumeroPersone()
getStatoCabina()
getStatoPorte()
getDirezione()
getStatoErrore()
```

In JML, un metodo dichiarato pure non modifica lo stato dell'oggetto e può quindi essere utilizzato all'interno delle specifiche.

8.2 Gestione delle richieste

La classe permette di aggiungere tre tipi di richieste:

```
aggiungiRichiestaInterna(int piano)
aggiungiChiamataSalita(int piano)
aggiungiChiamataDiscesa(int piano)
```

Questi metodi rappresentano rispettivamente:

- la pressione di un pulsante interno alla cabina;
- una chiamata esterna verso l'alto;
- una chiamata esterna verso il basso.

Tutti questi metodi richiedono che il piano sia valido e che il sistema non sia in stato di guasto:

```
//@ requires pianoValido(piano);
//@ requires statoErrore != StatoErrore.GUASTO;
//@ ensures richiesteAttive[indiceDelPiano(piano)] == true;
public void aggiungiRichiestaInterna(int piano)
```

In stato GUASTO, quindi, non vengono acquisite nuove richieste.

8.3 Gestione persone e sovraccarico

`aggiornaPersone(int personeEntrate, int personeUscite)`

aggiorna il numero di persone presenti in cabina.

Le precondizioni richiedono che:

```
//@ requires personeEntrate >= 0;
//@ requires personeUscite >= 0;
//@ requires statoPorte == StatoPorte.APERTE;
//@ requires statoCabina == StatoCabina.FERMA || statoCabina == StatoCabina.BLOCCATA;
```

Quindi il numero di persone può essere modificato solo se le porte sono aperte e la cabina è ferma o bloccata.

La postcondizione garantisce che il numero di persone non diventi mai negativo:

```
//@ ensures numeroPersone >= 0;
```

Il metodo è annotato con:

```
@CodeBigintMath
```

Questa annotazione viene usata per evitare falsi allarmi legati all'overflow degli interi durante la verifica statica con ESC.

Il metodo:

`gestisciSovraccarico()`

gestisce il passaggio allo stato OVERLOAD.

Se il numero di persone supera la capacità massima, il sistema entra in sovraccarico:

```
statoErrore = OVERLOAD
statoCabina = BLOCCATA
statoPorte = APERTE
direzione = NESSUNA
timer = 0
```

Se invece il sistema era in OVERLOAD e il numero di persone torna entro la capacità massima, il sistema ritorna operativo:

```
statoErrore = NESSUNO
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 0
```

8.4 Gestione guasto e timer

Il metodo:

```
attivaGuasto()
```

attiva lo stato di guasto tecnico.

Le precondizioni richiedono che il sistema non sia già in errore:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
```

Quando viene attivato il guasto, il sistema passa nello stato:

```
statoErrore = GUASTO
statoCabina = BLOCCATA
statoPorte = CHIUSE
direzione = NESSUNA
timer = TIMER_MASSIMO
```

Il metodo:

```
gestisciTimerGuasto()
```

viene utilizzato quando il sistema si trova nello stato GUASTO.

Se il timer è maggiore di zero, viene decrementato:

```
timer = timer - 1;
```

Se invece il timer è uguale a zero, il sistema viene ripristinato:

```
statoErrore = NESSUNO
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 0
```

Il contratto JML descrive formalmente entrambi i casi.

8.5 Gestione porte, servizio del piano e stato di attesa

Il metodo:

```
chiudiPorte()
```

chiude le porte dell'ascensore.

Può essere eseguito solo se:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
//@ requires statoCabina == StatoCabina.FERMA;
//@ requires statoPorte == StatoPorte.APERTE;
```

Quindi le porte vengono chiuse solo se il sistema è operativo, la cabina è ferma e le porte sono aperte.

Il metodo:

```
serviPianoCorrente()
```

serve la richiesta associata al piano corrente.

Le postcondizioni garantiscono che:

```
statoCabina = FERMA
statoPorte = APERTE
direzione = NESSUNA
richiesta del piano corrente = false
```

Il metodo rappresenta il momento in cui l'ascensore arriva al piano richiesto, apre le porte e rimuove la richiesta attiva.

Il metodo:

```
mettiInAttesa()
```

porta l'ascensore nello stato di attesa.

Il metodo richiede che non siano presenti errori:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
```

Le postcondizioni garantiscono:

```
statoCabina = FERMA
direzione = NESSUNA
```

Questo metodo rappresenta il comportamento dell'ascensore quando non ci sono richieste da servire.

8.6 Ricerca richieste, scelta direzione e movimento

La classe contiene tre metodi per analizzare le richieste attive:

```
esisteRichiesta()
esisteRichiestaSopra()
esisteRichiestaSotto()
```

Questi metodi sono dichiarati pure, perché leggono lo stato ma non lo modificano.

Il metodo:

```
esisteRichiesta()
```

verifica se esiste almeno una richiesta attiva in qualsiasi piano.

La postcondizione JML è:

```
//@ ensures \result <==> (\exists int i; 0 <= i && i < NUMERO_PIANI; richiesteAttive[i]);
```

Il ciclo è annotato con invarianti di ciclo:

```
//@ loop_invariant 0 <= i && i <= NUMERO_PIANI;
//@ loop_invariant (\forall int j; 0 <= j && j < i; richiesteAttive[j] == false);
```

Il metodo:

`esisteRichiestaSopra()`

verifica se esiste una richiesta attiva a un piano superiore rispetto al piano corrente.

La postcondizione JML è:

```
//@ ensures \result <==>
//@     (\exists int p; pianoCorrente < p && p <= PIANO_MASSIMO;
//@         richiesteAttive[indiceDelPiano(p)]);
```

Il metodo:

`esisteRichiestaSotto()`

verifica se esiste una richiesta attiva a un piano inferiore rispetto al piano corrente.

La postcondizione JML è:

```
//@ ensures \result <==>
//@     (\exists int p; PIANO_MINIMO <= p && p < pianoCorrente;
//@         richiesteAttive[indiceDelPiano(p)]);
```

Il metodo:

`scegliDirezione()`

decide la direzione dell'ascensore in base alle richieste attive.

Il metodo può essere eseguito solo se il sistema non presenta errori:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
```

La logica implementata è la seguente:

- se non esistono richieste, la direzione diventa NESSUNA;
- se la direzione corrente è SU e ci sono richieste sopra, la direzione resta SU;
- se la direzione corrente è SU ma non ci sono richieste sopra, il sistema cerca richieste sotto;
- se la direzione corrente è GIU e ci sono richieste sotto, la direzione resta GIU;
- se la direzione corrente è GIU ma non ci sono richieste sotto, il sistema cerca richieste sopra;

- se la direzione corrente è NESSUNA, il sistema sceglie una direzione in base alle richieste disponibili.

Il metodo:

```
muoviDiUnPiano()
```

sposta l'ascensore di un piano nella direzione corrente.

Può essere eseguito solo se:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
//@ requires statoPorte == StatoPorte.CHIUSE;
```

Quindi l'ascensore può muoversi solo senza errori e con le porte chiuse.

Se la direzione è SU e il piano corrente è minore del piano massimo, il piano viene incrementato:

```
pianoCorrente = pianoCorrente + 1;
statoCabina = StatoCabina.IN_MOVIMENTO;
```

Se la direzione è GIU e il piano corrente è maggiore del piano minimo, il piano viene decrementato:

```
pianoCorrente = pianoCorrente - 1;
statoCabina = StatoCabina.IN_MOVIMENTO;
```

Se non è possibile muoversi, la cabina viene fermata e la direzione viene annullata.

9. Classe InputAscensore

La classe *InputAscensore* rappresenta gli input esterni ricevuti dal sistema durante un singolo passo logico

Al suo interno vengono memorizzate eventuali richieste interne, chiamate esterne di salita o discesa, l'evento di guasto e il numero di persone entrate o uscite dalla cabina.

Sono inoltre presenti metodi di supporto che permettono di verificare se una richiesta è effettivamente presente:

```
haRichiestaInterna()
haChiamataSalita()
haChiamataDiscesa()
```

La classe *InputAscensore* non modifica direttamente lo stato dell'ascensore. Il suo compito è raccogliere gli input esterni, che vengono poi interpretati dalla classe *ControlloreAscensore*.

10. Classe ControlloreAscensore

La classe *ControlloreAscensore* ha il compito di coordinare l'evoluzione del sistema a ogni passo logico. Mentre la classe *Ascensore* contiene lo stato interno e i metodi che modificano tale stato, il controllore stabilisce l'ordine con cui questi metodi devono essere invocati.

Questa separazione permette di mantenere distinta la rappresentazione dello stato dalla logica di coordinamento, rendendo il codice più leggibile e più vicino alla struttura del modello ASM, nel quale una regola principale coordina l'esecuzione delle singole regole.

Il metodo principale della classe è

```
public void eseguiPasso(InputAscensore input)
```

A ogni passo logico, il controllore esegue quattro operazioni principali:

```
acquisisciRichieste(input);
gestisciPersone(input);
gestisciErrore(input);
gestisciComportamentoNormale();
```

In questo modo *ControlloreAscensore* svolge un ruolo simile alla regola principale del modello ASM, perché coordina le varie operazioni elementari e determina l'evoluzione complessiva del sistema.

11. Interfaccia web dimostrativa

Oltre al nucleo logico del sistema, il progetto include una semplice interfaccia web dimostrativa realizzata tramite HttpServer di Java.

L'interfaccia web ha lo scopo di rendere osservabile il comportamento del nucleo Java dell'ascensore. Non introduce una nuova logica di gestione del sistema, ma utilizza direttamente le classi già presenti nell'implementazione:

```
Ascensore
ControlloreAscensore
InputAscensore
```

La classe AscensoreHttpServer espone tre endpoint principali:

- /: utilizzato per restituire la pagina HTML dell'interfaccia;
- /stato: utilizzato per ottenere lo stato corrente dell'ascensore in formato JSON;
- /azione: utilizzato per inviare comandi al sistema, come richieste interne, chiamate esterne, aggiornamento del numero di persone, guasto, reset e gestione della simulazione automatica.

L'interfaccia supporta due modalità operative:

- simulazione automatica;
- controllo manuale.

Nella modalità automatica il sistema genera periodicamente richieste, ingressi e uscite di persone e possibili guasti. Nella modalità manuale l'utente può invece inserire direttamente richieste interne, chiamate esterne, variazioni del numero di persone e guasti.

La pagina web mostra dinamicamente:

- piano corrente;
- numero di persone;
- stato della cabina;
- stato delle porte;
- direzione;
- stato di errore;
- stato della simulazione automatica;
- richieste attive;
- log degli eventi principali;
- rappresentazione grafica semplificata della cabina.

12. Test JUnit

Oltre alla verifica statica tramite JML/OpenJML, il progetto include una suite di test *JUnit* per verificare il comportamento eseguibile della parte Java.

La suite verifica i principali scenari operativi del sistema, tra cui:

- stato iniziale e comportamento in assenza di richieste;
- acquisizione di richieste interne ed esterne;
- movimento verso piani superiori e inferiori;
- servizio del piano corrente e gestione delle porte;
- gestione del numero di persone;
- sovraccarico e risoluzione del sovraccarico;
- guasto tecnico e ripristino;
- blocco delle nuove richieste durante il guasto;
- conservazione delle richieste già acquisite;
- casi limite, come input nulli, richieste non valide, valori negativi e movimento ai piani estremi.

I test sono stati eseguiti sia in ambiente *Eclipse* tramite *JUnit*, sia automaticamente nella pipeline *GitHub Actions*. La copertura del codice è stata analizzata tramite *EclEmma*.

13. Test Selenium

Oltre ai test JUnit sul nucleo logico, il progetto include test Selenium per verificare il corretto funzionamento dell'interfaccia web dimostrativa.

I test Selenium sono test di integrazione, perché verificano il collegamento tra:

- pagina web
- server HTTP
- ControlloreAscensore
- Ascensore

Prima di ogni test viene eseguito un reset dello stato del sistema, così da rendere i test indipendenti tra loro. Al termine dell'esecuzione, il browser viene chiuso e il server viene arrestato.

I test Selenium verificano in particolare:

- apertura corretta della pagina web;
- visualizzazione dello stato iniziale dell'ascensore;
- passaggio alla modalità di controllo manuale;
- inserimento di una richiesta interna tramite interfaccia;
- aggiornamento della sezione delle richieste attive;
- reset dei campi di input manuali dopo l'invio;
- avvio e arresto della simulazione automatica;
- disabilitazione del controllo manuale durante la simulazione automatica;
- visualizzazione dello stato di guasto;
- visualizzazione dello stato di sovraccarico.

In locale i test utilizzano il ChromeDriver presente nella cartella drivers, mentre nella pipeline GitHub Actions ChromeDriver viene configurato automaticamente per l'ambiente Linux del runner. In CI i test vengono eseguiti in modalità headless.

Questi test non sostituiscono i test JUnit sul nucleo logico, ma verificano che l'interfaccia web sia correttamente collegata alla logica Java del sistema.

14. Verifica tramite ESC/OpenJML

La verifica statica è stata eseguita mediante ESC di OpenJML.

L'obiettivo della verifica è controllare che i metodi della classe rispettino le specifiche JML definite tramite:

- invarianti di classe;
- precondizioni;
- postcondizioni;
- annotazioni sui cicli;
- metodi pure.

La verifica ha permesso di controllare la coerenza del codice rispetto alle proprietà dichiarate e di rafforzare la gestione degli stati di errore.

Alcuni metodi, come scegliDirezione(), presentano una logica decisionale più articolata e possono risultare più difficili da dimostrare completamente tramite verifica automatica. Questo non indica necessariamente un errore nel comportamento, ma rappresenta un limite pratico della verifica statica.

Per questo motivo, la verifica statica tramite JML è stata affiancata da test JUnit sul comportamento eseguibile e da test Selenium per verificare l'integrazione dell'interfaccia web con il nucleo Java.

15. Conclusione

L'implementazione Java della classe Ascensore rappresenta il nucleo logico principale del sistema ascensore modellato in ASM.

La specifica JML permette di formalizzare le principali proprietà di sicurezza e coerenza dello stato, tra cui la validità del piano corrente, la validità del timer, la non negatività del numero di persone, l'assenza di valori nulli negli stati, il movimento solo con porte chiuse e la corretta gestione degli stati di guasto e sovraccarico.

Il codice Java non sostituisce il modello ASM completo, ma ne implementa una parte significativa in forma eseguibile e verificabile.

La parte Java/JML può quindi essere considerata coerente con il modello formale e adeguata come nucleo implementativo del progetto. I test JUnit completano questa verifica controllando il comportamento operativo del sistema nei principali scenari, affiancando la verifica statica svolta tramite OpenJML.

L'interfaccia web dimostrativa permette inoltre di osservare dinamicamente il comportamento del nucleo Java e di interagire con il sistema in modalità automatica o manuale. Poiché utilizza direttamente Ascensore, ControlloreAscensore e InputAscensore, essa rimane coerente con la logica implementativa verificata.

I test Selenium completano questa parte controllando che pagina web, server HTTP e nucleo logico Java interagiscano correttamente.

Infine, i test automatici JUnit e Selenium sono stati integrati in un workflow GitHub Actions, così da automatizzare la compilazione del progetto e l'esecuzione dei test a ogni modifica del repository.